```
"""
============================================================
  Limkokwing Library Management System
  Module  : PROG315 - Object-Oriented Programming 2
  Author  : [Your Full Name]
  Date    : May 2026
============================================================
"""

import asyncio
import time
from datetime import datetime, timedelta
from typing import Optional

# ──────────────────────────────────────────────────────────
#  DATA STORE  (simulates a relational database)
# ──────────────────────────────────────────────────────────
books: dict[str, dict] = {
    "B001": {"id": "B001", "title": "Clean Code",
             "author": "Robert C. Martin",
             "category": "Software Engineering", "available": True},
    "B002": {"id": "B002", "title": "Python Crash Course",
             "author": "Eric Matthes",
             "category": "Programming", "available": True},
    "B003": {"id": "B003", "title": "Design Patterns",
             "author": "Gang of Four",
             "category": "Software Engineering", "available": False},
    "B004": {"id": "B004", "title": "Artificial Intelligence",
             "author": "Stuart Russell",
             "category": "Artificial Intelligence", "available": True},
}

loans: dict[str, dict] = {
    "L001": {
        "loan_id":    "L001",
        "user_id":    "U003",
        "book_id":    "B003",
        "borrow_date": (datetime.now() - timedelta(days=20)).isoformat(),
        "due_date":    (datetime.now() - timedelta(days=6)).isoformat(),
        "returned":    False,
    }
}

FINE_PER_DAY: int = 5000   # Le 5,000 per overdue day
```

```
# ─────────────────────────────────────────────────────────
#  SECTION 1 — SYNCHRONOUS BUSINESS LOGIC HELPERS
#  Plain def functions; each runs to completion in order.
#  All data validation and state changes live here.
# ─────────────────────────────────────────────────────────

def sync_search_books(
    title:    Optional[str],
    author:   Optional[str],
    category: Optional[str],
) -> dict:
    """
    Search the library catalogue.
    All three parameters are optional; omitting all returns
    the full catalogue.
    """
    results: list[dict] = list(books.values())

    if title:
        results = [b for b in results
                   if title.lower() in b["title"].lower()]
    if author:
        results = [b for b in results
                   if author.lower() in b["author"].lower()]
    if category:
        results = [b for b in results
                   if category.lower() in b["category"].lower()]

    return {"status": "success", "count": len(results), "books": results}


def sync_borrow_book(user_id: str, book_id: str) -> dict:
    """
    Create a loan record for an available book.
    Raises ValueError if the book does not exist or is on loan.
    """
    if book_id not in books:
        raise ValueError(f"Book {book_id} does not exist.")

    book: dict = books[book_id]

    if not book["available"]:
        raise ValueError(f"'{book['title']}' is currently on loan.")

    book["available"] = False
```

```
    loan_id: str        = f"L{len(loans) + 1:03d}"
    borrow_dt: datetime = datetime.now()
    due_dt: datetime    = borrow_dt + timedelta(days=14)

    loan: dict = {
        "loan_id":     loan_id,
        "user_id":     user_id,
        "book_id":     book_id,
        "book_title":  book["title"],
        "borrow_date": borrow_dt.isoformat(),
        "due_date":    due_dt.isoformat(),
        "returned":    False,
    }
    loans[loan_id] = loan

    return {
        "status":  "success",
        "message": f"'{book['title']}' borrowed successfully.",
        "loan":    loan,
    }


def sync_return_book(loan_id: str, user_id: str) -> dict:
    """
    Close an active loan and calculate any overdue fine.
    Fine rate: Le 5,000 per day past the due date.
    Raises ValueError on invalid loan, user mismatch, or
    already-returned book.
    """
    if loan_id not in loans:
        raise ValueError(f"Loan {loan_id} not found.")

    loan: dict = loans[loan_id]

    if loan["user_id"] != user_id:
        raise ValueError("User ID does not match the borrower on record.")

    if loan["returned"]:
        raise ValueError("This book has already been returned.")

    loan["returned"] = True
    books[loan["book_id"]]["available"] = True

    due_dt: datetime = datetime.fromisoformat(loan["due_date"])
    days_late: int   = max(0, (datetime.now() - due_dt).days)
    total_fine: int  = days_late * FINE_PER_DAY
```

```
    return {
        "status":       "success",
        "message":      "Book returned successfully.",
        "days_overdue": days_late,
        "fine_leones":  total_fine,
    }


def sync_overdue_loans() -> dict:
    """
    Return all active loans whose due date has passed,
    along with the accumulated fine for each.
    """
    now: datetime       = datetime.now()
    overdue: list[dict] = []

    for loan in loans.values():
        if loan["returned"]:
            continue
        due: datetime = datetime.fromisoformat(loan["due_date"])
        days: int     = (now - due).days
        if days > 0:
            overdue.append({
                **loan,
                "days_overdue": days,
                "fine_leones":  days * FINE_PER_DAY,
            })

    return {
        "status":        "success",
        "total_overdue": len(overdue),
        "overdue_loans": overdue,
    }


# ──────────────────────────────────────────────────────────
#  SECTION 2 — ASYNCHRONOUS ENDPOINT HANDLERS
#  async def + await lets the event loop handle many
#  requests concurrently without blocking.
#  Each await point yields control so other coroutines
#  can proceed while simulated I/O completes.
# ──────────────────────────────────────────────────────────

async def get_books(
    title:    Optional[str] = None,
    author:   Optional[str] = None,
```

```
    category: Optional[str] = None,
) -> dict:
    """Async handler for GET /books."""
    await asyncio.sleep(0.05)          # simulate async DB read
    return sync_search_books(title, author, category)


async def borrow_book(user_id: str, book_id: str) -> dict:
    """Async handler for POST /borrow."""
    await asyncio.sleep(0.08)          # simulate async DB write
    return sync_borrow_book(user_id, book_id)


async def return_book(loan_id: str, user_id: str) -> dict:
    """Async handler for PUT /return."""
    await asyncio.sleep(0.05)          # simulate async DB update
    return sync_return_book(loan_id, user_id)


async def get_overdue() -> dict:
    """Async handler for GET /overdue."""
    await asyncio.sleep(0.05)          # simulate async DB scan
    return sync_overdue_loans()


# ──────────────────────────────────────────────────────────
#  SECTION 3 — CONCURRENT SIMULATION
#  asyncio.gather() fires all coroutines simultaneously.
#  All requests are launched before any single one finishes,
#  demonstrating true concurrent handling.
# ──────────────────────────────────────────────────────────

async def async_user_borrow(user_id: str, book_id: str) -> str:
    """Simulate one user's borrow request asynchronously."""
    print(f"  [{user_id}]  --> Requesting book {book_id} ...")
    await asyncio.sleep(0.15)
    try:
        result: dict = sync_borrow_book(user_id, book_id)
        return (
            f"  [{user_id}]  SUCCESS - Borrowed"
            f" \"{result['loan']['book_title']}\""
            f"  |  Loan: {result['loan']['loan_id']}"
            f"  |  Due : {result['loan']['due_date'][:10]}"
        )
    except ValueError as e:
        return f"  [{user_id}]  FAILED  - {e}"
```

```
async def async_user_return(loan_id: str, user_id: str) -> str:
    """Simulate one user's return request asynchronously."""
    print(f"  [{user_id}]  <-- Returning loan {loan_id} ...")
    await asyncio.sleep(0.10)
    try:
        result: dict = sync_return_book(loan_id, user_id)
        fine: str = (
            f"Le {result['fine_leones']:,}"
            if result["fine_leones"] > 0 else "None"
        )
        return (
            f"  [{user_id}]  SUCCESS - Returned loan {loan_id}"
            f"  |  Days overdue: {result['days_overdue']}"
            f"  |  Fine: {fine}"
        )
    except ValueError as e:
        return f"  [{user_id}]  FAILED  - {e}"


def divider(label: str) -> None:
    print("\n" + "=" * 62)
    print(f"  {label}")
    print("=" * 62)


async def run_simulation() -> None:
    """
    Entry point for the standalone simulation.
    Demonstrates synchronous sequential calls first,
    then asynchronous concurrent calls using asyncio.gather().
    """
    print("\n" + "=" * 62)
    print("  Limkokwing Library Management System")
    print("  PROG315 | Object-Oriented Programming 2")
    print("=" * 62)

    # ── Synchronous demo 1: search catalogue ──────────────
    divider("SYNC  |  GET /books  -->  category=Programming")
    result: dict = sync_search_books(None, None, "Programming")
    print(f"  Status  : {result['status']}")
    print(f"  Results : {result['count']} book(s) found")
    for b in result["books"]:
        avail: str = "Available" if b["available"] else "On Loan"
        print(f"    [{b['id']}]  {b['title']}  |  {b['author']}  |  {avail}")

    # ── Synchronous demo 2: overdue check ─────────────────
```

```
    divider("SYNC  |  GET /overdue")
    od: dict = sync_overdue_loans()
    if od["total_overdue"] == 0:
        print("  No overdue loans at this time.")
    for entry in od["overdue_loans"]:
        print(
            f"  Loan {entry['loan_id']}"
            f"  |  User: {entry['user_id']}"
            f"  |  Days overdue: {entry['days_overdue']}"
            f"  |  Fine: Le {entry['fine_leones']:,}"
        )

    # ── Asynchronous demo: 3 users borrow at once ────────
    divider("ASYNC  |  POST /borrow  -->  3 users simultaneously")
    t0: float = time.perf_counter()
    borrow_results: list[str] = await asyncio.gather(
        async_user_borrow("U010", "B001"),
        async_user_borrow("U011", "B002"),
        async_user_borrow("U012", "B004"),
    )
    elapsed: float = time.perf_counter() - t0
    print()
    for r in borrow_results:
        print(r)
    print(f"\n  All 3 requests completed in {elapsed:.3f}s  (concurrent)")

    # ── Asynchronous demo: 2 users return at once ────────
    divider("ASYNC  |  PUT /return  -->  2 users simultaneously")
    t0 = time.perf_counter()
    return_results: list[str] = await asyncio.gather(
        async_user_return("L001", "U003"),
        async_user_return("L002", "U010"),
    )
    elapsed = time.perf_counter() - t0
    print()
    for r in return_results:
        print(r)
    print(f"\n  Both returns completed in {elapsed:.3f}s  (concurrent)")

    divider("SIMULATION COMPLETE")
    print("  All synchronous and asynchronous operations passed.\n")


if __name__ == "__main__":
    asyncio.run(run_simulation())
```